

**TRƯỜNG ĐẠI HỌC GIAO THÔNG VẬN TẢI
PHẦN HIỆU TẠI TP.HCM
BỘ MÔN CÔNG NGHỆ THÔNG TIN**



Kiểm thử Hộp trắng — Kiểm thử luồng điều khiển

Buổi 3 / 12 — Môn: Kiểm thử Phần mềm

Học kỳ 2025–2026

Nội dung buổi học

- 1 Kiểm thử hộp trắng: khái niệm, so sánh với hộp đen
- 2 Đồ thị luồng điều khiển (Control Flow Graph — CFG)
- 3 Ví dụ trên DigiShield: `ageLabel()` và `evaluate()`
- 4 Các tiêu chí độ phủ: câu lệnh, nhánh, đường đi
- 5 Độ phức tạp cyclomatic $V(G)$ và basis path testing
- 6 Thiết kế bộ test đạt 100% độ phủ nhánh — đối chiếu test thật
- 7 Kiểm thử vòng lặp (loop testing)
- 8 Hạn chế của kiểm thử luồng điều khiển
- 9 Thực hành — Bài nộp

Vị trí trong đề cương môn học

Chương 3 — Kiểm thử hộp trắng (buổi 3 và buổi 4)

- 3.1. Giới thiệu
- 3.2. Khái niệm chung về kiểm thử hộp trắng
- 3.3. **Kiểm thử luồng điều khiển** ← **buổi hôm nay**
- 3.4. Độ phủ điều kiện, MC/DC, luồng dữ liệu, JaCoCo ← buổi 4

Buổi	Chủ đề
1–2	C1–C2: Tổng quan, quy trình
3–4	C3: Kiểm thử hộp trắng
5–7	C4: JUnit, Mockito, integration
8–9	C5: Kiểm thử hộp đen, API test
10–11	C6: Selenium
12	C7: Thực tiễn + bảo vệ BTL

Kiểm tra viết đầu giờ (10 phút)

Nội dung buổi 1–2: 7 nguyên tắc kiểm thử, V-model, STLC, mức kiểm thử. Điểm tích lũy cho cột **chuyên cần (10%)** và luyện dạng câu hỏi cho **bài kiểm tra giữa kỳ (10%)**.

Ôn tập buổi 2 — vì sao cần hộp trắng?

Buổi 2 đã học

- Quy trình kiểm thử (STLC), V-model
- Các mức kiểm thử: unit, integration, system, acceptance
- Testing pyramid: unit test là tầng đáy
- Đã giao Bài tập lớn theo nhóm trên DigiShield

Câu hỏi mở đầu

Nếu chỉ nhìn từ bên ngoài (nhập input, xem output), ta **không biết**:

- Nhánh code nào **chưa từng** được chạy?
- Có **dead code** không?
- Bộ test hiện tại “tốt” đến mức nào?

⇒ Cần nhìn **vào bên trong** mã nguồn: kiểm thử hộp trắng.

Kiểm thử hộp trắng là gì?

Định nghĩa

Kiểm thử hộp trắng (white-box testing) thiết kế test case **dựa trên cấu trúc bên trong** của phần mềm: mã nguồn, luồng điều khiển, luồng dữ liệu.^[1,2]

Tên gọi khác:

- Structural testing (kiểm thử cấu trúc)
- Glass-box / clear-box testing
- Code-based testing

Mục tiêu:

- Mọi câu lệnh được thực thi ít nhất một lần
- Mọi nhánh true/false đều được đi qua
- Phát hiện dead code, nhánh cài đặt sai
- **Đo được** chất lượng bộ test bằng độ phủ (coverage)

Hộp trắng vs Hộp đen

	Hộp trắng	Hộp đen
Cơ sở thiết kế test	Mã nguồn, CFG, luồng dữ liệu	Đặc tả, yêu cầu, giao diện
Cần biết code?	Có (đọc và hiểu code)	Không
Ai thực hiện	Developer, tester biết lập trình	Tester, BA, người dùng
Mức áp dụng chủ yếu	Unit test, integration test	System test, acceptance test
Phát hiện tốt	Nhánh sai, dead code, lỗi logic cài đặt	Thiếu chức năng, sai so với yêu cầu
Thước đo	Độ phủ câu lệnh / nhánh / đường đi	Độ phủ yêu cầu, lớp tương đương

Hai kỹ thuật bổ sung cho nhau: hộp trắng trả lời “code **đã viết** có được test đủ chưa?”; hộp đen trả lời “phần mềm có làm **đúng yêu cầu** không?”.

Khi nào dùng kiểm thử hộp trắng?

Tình huống điển hình:

- Viết **unit test** cho service, hàm tiện ích (buổi 5–6 sẽ cài đặt bằng JUnit/Mockito)
- Code review: chỉ ra nhánh chưa được test
- Nâng độ phủ theo yêu cầu dự án (DigiShield đang ép line coverage $\geq$ 10%, mục tiêu 50%)
- Kiểm thử logic nghiệp vụ nhiều rẽ nhánh: chấm điểm rủi ro, quyết định can thiệp giao dịch

Trên DigiShield hôm nay

Ta sẽ phân tích 2 method **thật**:

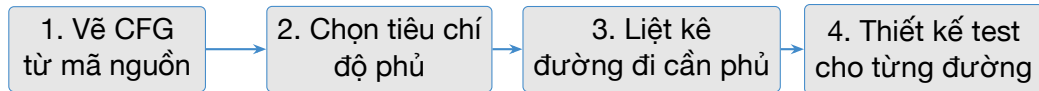
- `ReportingServiceImpl.ageLabel()` — nhãn tuổi báo cáo phishing
- `InterceptionServiceImpl.evaluate()` — quyết định chặn giao dịch đáng ngờ

và đối chiếu với **unit test thật** trong repo.

Kiểm thử luồng điều khiển — quy trình 4 bước

Control flow testing^[2]

Kỹ thuật hộp trắng chọn test case dựa trên các **đường thực thi** (execution path) qua mã nguồn.



- Bước 2: câu lệnh (statement), nhánh (branch), đường đi (path)...
- Bước 4: tìm **input** khiến chương trình đi đúng đường đã chọn.

Control Flow Graph — Định nghĩa

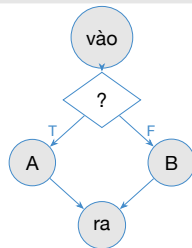
Định nghĩa

CFG là đồ thị có hướng $G = (N, E)$:

- N : tập **node** — mỗi node là một khối câu lệnh tuần tự (basic block)
- E : tập **cạnh có hướng** — mỗi cạnh là một bước chuyển điều khiển có thể xảy ra

Các loại node:

- **Entry node**: điểm vào duy nhất của method
- **Exit node**: điểm kết thúc
- **Node quyết định** (decision): có ≥ 2 cạnh ra — sinh từ `if`, `while`, `for`, `switch`
- **Node hợp** (junction): có ≥ 2 cạnh vào

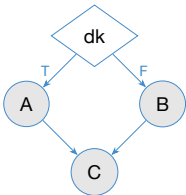


“?” = node quyết định; “ra” = node hợp.

Vẽ CFG từ cấu trúc rẽ nhánh

if có else

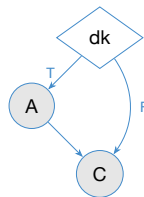
```
if (dk) { A; }  
else    { B; }  
C;
```



2 nhánh: T và F.

if KHÔNG có else

```
if (dk) { A; }  
C;
```

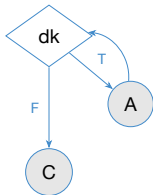


Nhánh F là cạnh “đi tắt” — **rất dễ quên khi test!**

Vẽ CFG từ vòng lặp và switch

while / for

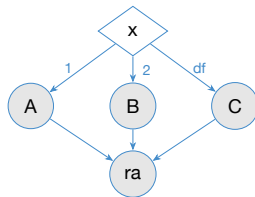
```
while (dk) { A; }  
C;
```



Cạnh quay lui (back edge) tạo **chu trình** $\Rightarrow$ số đường đi bùng nổ theo số lần lặp.

switch / if-else if-else

```
switch (x) {  
  case 1: A; break;  
  case 2: B; break;  
  default: C;  
}
```



Node quyết định có thể có **nhiều hơn 2** cạnh ra.

Quy ước vẽ CFG trong môn học

- 1 Gộp các câu lệnh tuần tự liên tiếp thành **một node** (basic block) — đồ thị gọn, không mất thông tin.
- 2 Mỗi điều kiện rẽ nhánh vẽ bằng **hình thoi**, ghi rõ cạnh **T** (true) và **F** (false).
- 3 Method có nhiều return: nối tất cả điểm return về **một exit node chung** — để công thức $V(G) = E - N + 2$ cho kết quả đúng.
- 4 Đánh **số thứ tự** node để mô tả đường đi, ví dụ: $1 \rightarrow 2(T) \rightarrow 3 \rightarrow 7$.

Lỗi hay gặp của sinh viên

Quên nhánh F của `if` không có `else`; quên exit node chung khi method có nhiều return; vẽ mỗi dòng code một node làm đồ thị rối.

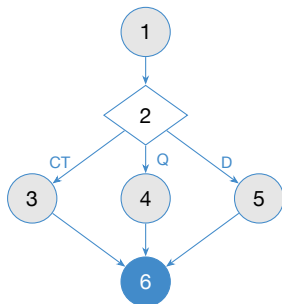
Khởi động: triage() — duyệt báo cáo phishing

ReportingServiceImpl.triage() — rút gọn từ code thật

```
1 public PhishingReport triage(UUID reportId,  
2                             TriageDecision decision) {  
3     PhishingReport report = ...;    // tìm báo cáo (node 1)  
4     switch (decision) {            // node 2 - quyết định  
5         case CONFIRM_THREAT -> {    // node 3  
6             report.setStatus(ReportStatus.CONFIRMED);  
7             eventPublisher.publish(...);  
8             return reportRepository.save(report); }  
9         case QUARANTINE -> {         // node 4  
10            report.setStatus(ReportStatus.QUARANTINED);  
11            return reportRepository.save(report); }  
12        case DISMISS -> {           // node 5  
13            report.setStatus(ReportStatus.DISMISSED);  
14            return reportRepository.save(report); }  
15    }  
16 }
```

Nhiệm vụ: analyst xác nhận đe dọa (CONFIRMED, phát sự kiện), tạm giữ để leo thang (QUARANTINED, **không** phát sự kiện), hoặc bác bỏ (DISMISSED).

CFG của `triage()`



Node	Nội dung
1	Tìm báo cáo theo reportId
2	switch (decision)
3	CT: THREAT + CONFIRMED + publish
4	Q: THREAT + QUARANTINED
5	D: CLEAN + DISMISSED
6	save + return (exit)

Node	Nội dung
1	Tìm báo cáo theo reportId
2	switch (decision)
3	CT: THREAT + CONFIRMED + publish
4	Q: THREAT + QUARANTINED
5	D: CLEAN + DISMISSED
6	save + return (exit)

Ba đường đi qua node 3, 4, 5 $\Rightarrow$ tối thiểu **3 test case**.

Nhánh ẩn — điểm thảo luận

Trong code thật, `findById(...).orElseThrow(...)` còn một nhánh nữa: **không tìm thấy báo cáo** $\rightarrow$ ném exception. Nhánh ẩn trong API cũng phải được test!

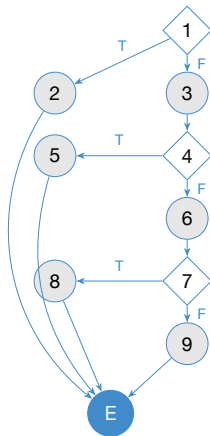
Ví dụ chủ lực 1: ageLabel() — code thật

ReportingServiceImpl.ageLabel() — nhãn tuổi báo cáo (“2p”, “3h”, “1d”)

```
1 private String ageLabel(Instant reportedAt, Instant now) {  
2     if (reportedAt == null) {                // node 1  
3         return null;                        // node 2  
4     }  
5     Duration d = Duration.between(reportedAt, now);  
6     long mins = Math.max(0, d.toMinutes()); // node 3  
7     if (mins < 60) {                        // node 4  
8         return mins + "p";                 // node 5 - "N phut"  
9     }  
10    long hours = d.toHours();                // node 6  
11    if (hours < 24) {                       // node 7  
12        return hours + "h";                // node 8 - "N gio"  
13    }  
14    return d.toDays() + "d";                // node 9 - "N ngay"  
15 }
```

Hàm **thuần** (pure): không phụ thuộc DB $\Rightarrow$ dễ test. 3 điểm quyết định, 4 kết quả loại trừ nhau.

CFG của ageLabel()



Node	Nội dung
1	if (reportedAt == null)
2	return null
3	tính d, mins
4	if (mins < 60)
5	return mins + "p"
6	tính hours
7	if (hours < 24)
8	return hours + "h"
9	return days + "d"
E	exit chung (quy ước)

Đếm: $N = 10$ node, $E = 12$ cạnh, **3 node quyết định** (1, 4, 7), **4 đường đi** từ vào đến ra.

ageLabel() — 4 đường đi và test data

Path	Đường đi	Test data (reportedAt so với now)	Expected
P1	$1(T) \rightarrow 2 \rightarrow E$	reportedAt = null	null
P2	$1(F) \rightarrow 3 \rightarrow 4(T) \rightarrow 5 \rightarrow E$	30 phút trước	"30p"
P3	$1(F) \rightarrow 3 \rightarrow 4(F) \rightarrow 6 \rightarrow 7(T) \rightarrow 8 \rightarrow E$	2 giờ trước	"2h"
P4	$1(F) \rightarrow 3 \rightarrow 4(F) \rightarrow 6 \rightarrow 7(F) \rightarrow 9 \rightarrow E$	3 ngày trước	"3d"

Nhận xét

- 4 test case $\Rightarrow$ đi hết mọi node, mọi cạnh, mọi đường đi.
- Giá trị **biên** 59/60 phút, 23/24 giờ chưa nằm trong bảng — đó là việc của BVA (buổi 8). Hộp trắng và hộp đen bổ sung nhau!

Ví dụ chủ lực 2: bối cảnh nghiệp vụ evaluate()

Module interception — can thiệp giao dịch đáng ngờ

Nạn nhân lừa đảo qua điện thoại thường chuyển tiền **trong khi vẫn đang nghe kẻ lừa đảo thao túng**. DigiShield đánh giá mỗi giao dịch theo 3 tín hiệu:

- onCall — người dùng đang trong cuộc gọi?
- newPayee — chuyển cho người thụ hưởng mới?
- watchlistHit — tài khoản đích nằm trong danh sách theo dõi?

Luật quyết định (code thật)

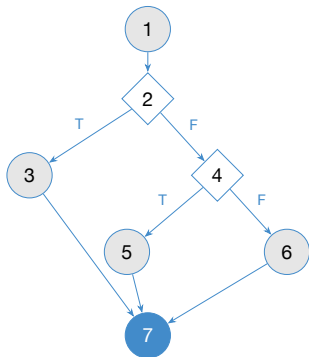
- Cả 3 tín hiệu cùng bật ⇒ **PAUSE** (tạm dừng giao dịch, hiện cảnh báo giáo dục)
- Chỉ trúng watchlist ⇒ **WARN** (cảnh báo nhưng cho phép)
- Còn lại ⇒ **ALLOW** (cho qua)

API: POST /api/v1/interventions/evaluate

evaluate() — code thật (rút gọn)

```
1 public InterventionDecision evaluate(EvaluateRequest req) {
2     List<String> signals = new ArrayList<>();
3     if (req.onCall()) signals.add("ON_CALL"); // S1
4     if (req.newPayee()) signals.add("NEW_PAYEE"); // S2
5     boolean watchlistHit = watchRepository
6         .findByTenantIdAndValue(tenantId, req.destAccount())
7         .isPresent();
8     if (watchlistHit) signals.add("WATCHLIST_HIT"); // S3
9     Decision decision;
10    if (req.onCall() && req.newPayee() && watchlistHit) {
11        decision = Decision.PAUSE; // D1
12    } else if (watchlistHit) {
13        decision = Decision.WARN; // D2
14    } else {
15        decision = Decision.ALLOW;
16    }
17    eventRepository.save(event); // ghi su kien can thiep
18    return new InterventionDecision(decision.name(),
19        signals, message);
20 }
```

CFG của evaluate() – lỗi quyết định



Gộp phần gom tín hiệu (S1–S3) vào node 1 để tập trung vào lỗi quyết định; sẽ “mở” nó ra ở phần sau.

Node	Nội dung
1	Gom signals, tra watchlist
2	onCall && newPayee && hit?
3	decision = PAUSE
4	watchlistHit?
5	decision = WARN
6	decision = ALLOW
7	save + return (exit)

Đếm: $N = 7$, $E = 8$, 2 node quyết định (2, 4).

3 đường đi:

- 1 → 2(T) → 3 → 7 — PAUSE
- 1 → 2(F) → 4(T) → 5 → 7 — WARN
- 1 → 2(F) → 4(F) → 6 → 7 — ALLOW

Độ phủ câu lệnh (Statement Coverage — SC)

Định nghĩa^[1,3]

$$SC = \frac{\text{số câu lệnh đã được thực thi bởi bộ test}}{\text{tổng số câu lệnh thực thi được}} \times 100\%$$

Đạt 100% khi **mỗi câu lệnh** (mỗi node của CFG) được thực thi ít nhất một lần.

- Là tiêu chí **yếu nhất** — mức tối thiểu mà mọi bộ unit test nên đạt cho code nghiệp vụ.
- Phát hiện được: dead code, câu lệnh ném exception chưa từng chạy.
- Công cụ đo tự động: JaCoCo (buổi 4) — hôm nay ta **tính tay** trên CFG để hiểu bản chất.

Tính tay SC trên CFG của evaluate()

Test	Node đi qua	Node đã phủ	SC
T1: PAUSE (cả 3 tín hiệu)	1, 2, 3, 7	4/7	57%
+ T2: WARN (chỉ watchlist)	thêm 4, 5	6/7	86%
+ T3: ALLOW (không tín hiệu)	thêm 6	7/7	100%

Kết luận

Cần tối thiểu **3 test case** để đạt 100% SC — vì ba node 3, 5, 6 nằm trên ba nhánh **loại trừ nhau**, không test nào đi qua hai node cùng lúc.

Câu hỏi nhanh: với `ageLabel()`, cần bao nhiêu test để đạt 100% SC? (Trả lời: 4 — bốn return loại trừ nhau.)

Độ phủ nhánh (Branch/Decision Coverage — BC)

Định nghĩa^[1,3]

$$BC = \frac{\text{số nhánh đã được thực thi}}{\text{tổng số nhánh}} \times 100\%$$

Mỗi node quyết định (`if`, `while`, `for`, `switch`) phải có **mọi cạnh ra** (cả T lẫn F) được đi qua ít nhất một lần.

Mạnh hơn SC vì:

- 100% BC $\Rightarrow$ 100% SC (đi hết cạnh thì hết node)
- Chiều ngược lại **không đúng**: `if` không có `else` — SC bỏ sót nhánh F

Ngưỡng thực tế

Safety-critical	100%
Fintech / ngân hàng	80–90%
SonarQube mặc định	80%
Yêu cầu BTL môn học	70%

Tính tay BC trên CFG của evaluate()

Lỗi quyết định có 2 node quyết định $\Rightarrow$ **4 nhánh**:

Nhánh	Ý nghĩa	T1 PAUSE	T2 WARN	T3 ALLOW
Node 2 — T	cả 3 tín hiệu bật	✓		
Node 2 — F	thiếu ít nhất 1 tín hiệu		✓	✓
Node 4 — T	trúng watchlist		✓	
Node 4 — F	không trúng watchlist			✓

Kết luận

Bộ 3 test {PAUSE, WARN, ALLOW} phủ 4/4 nhánh $\Rightarrow$ **100% BC** trên lỗi quyết định. Ở đây SC và BC cùng cần 3 test — vì cấu trúc if--else if--else luôn có “else” tường minh.

Vì sao 100% SC vẫn chưa đủ?

if không else — biến thể rút gọn của `StubApiClient.moderate()`

```
String verdict = "flag";  
if (reasons.size() >= 2) {  
    verdict = "block";  
}
```

Test duy nhất `reasons.size() = 2`:

- Mọi câu lệnh chạy $\Rightarrow$ **SC = 100%**
- Nhánh F **chưa từng** được test!

Lỗi lọt lưới

Nếu dev viết nhầm điều kiện thành `size() >= 1`:

- Test `size=2`: vẫn ra "block" — PASS
- Chỉ test đi nhánh F (`size=1`, kỳ vọng "flag") mới lộ lỗi: code sai trả về "block"

$\Rightarrow$ BC bắt được lỗi mà SC bỏ qua.

Nhưng 100% BC cũng chưa phải “hết lỗi”

Trong `ageLabel()`, nếu `mins < 60` bị viết nhầm thành `mins <= 60`: bộ test 30 phút / 2 giờ / 3 ngày vẫn đạt 100% BC mà không lộ lỗi tại đúng **biên 60 phút**. Độ phủ cao là **cần** nhưng không **đủ** — cần kết hợp BVA (buổi 8).

Độ phủ đường đi (Path Coverage)

Định nghĩa

Mỗi **đường đi** (path) khả thi từ entry đến exit của CFG phải được thực thi ít nhất một lần.

Trên hai ví dụ hôm nay:

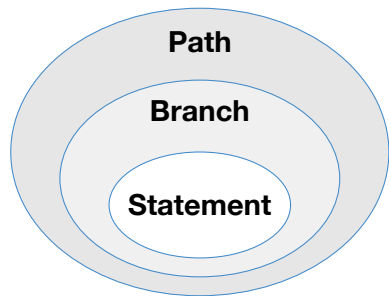
- `evaluate()` (lỗi): **3** đường đi
- `ageLabel()`: **4** đường đi
- Cả hai: path coverage đạt được với đúng bộ test của branch coverage
 - vì **không có vòng lặp** và các `if` nối tiếp loại trừ nhau

Vì sao path coverage bất khả thi?

- `k if` độc lập nối tiếp: 2^k đường đi
- Vòng lặp tối đa n lần: nhân số đường lên $n + 1$ lần
- `moderate()` duyệt 10 từ khóa: $2^{10} = 1024$ đường đi qua vòng lặp!

Giải pháp thực dụng: chỉ test một tập đường đi **độc lập tuyến tính** — basis path testing, dựa trên độ phức tạp cyclomatic $V(G)$ (phần tiếp theo).

Quan hệ bao hàm (subsumption) giữa các tiêu chí



Tiêu chí ngoài **bao hàm** (mạnh hơn) tiêu chí trong:
 $\text{Path} \supset \text{Branch} \supset \text{Statement}$

Tiêu chí	Phát hiện tốt	Chi phí
Statement	dead code	thấp
Branch	nhánh sai, thiếu else	trung bình
Path	tương tác giữa các quyết định	rất cao

Ghi nhớ

100% Path $\Rightarrow$ 100% Branch $\Rightarrow$ 100% Statement. Chiều ngược lại đều **sai**. Buổi 4 sẽ chèn thêm hai mức Condition và MC/DC vào giữa Branch và Path.

Độ phức tạp cyclomatic $V(G)$ — McCabe, 1976 (1/2)

Ba cách tính tương đương (đồ thị liên thông, 1 exit chung)^[5]

$$V(G) = E - N + 2 \quad V(G) = \text{số node quyết định} + 1 \quad V(G) = \text{số miền kín} + 1$$

E : số cạnh, N : số node của CFG; miền kín = region khi vẽ đồ thị phẳng.

Điều kiện áp dụng: “số node quyết định + 1” chỉ đúng khi mỗi node quyết định có đúng **2 cạnh ra**; với switch có k cạnh ra (slide 11) dùng

$$V(G) = \sum (\text{bậc ra (out-degree)} - 1) + 1.$$

Độ phức tạp cyclomatic $V(G)$ — McCabe, 1976 (2/2)

Ý nghĩa:

- = số đường đi **độc lập tuyến tính** tối đa
- = số test case **tối thiểu** cho basis path testing
- Thước đo độ phức tạp:
 $V(G) > 10 \Rightarrow$ method nên tách nhỏ

Lưu ý khi tính

Method nhiều return: phải nối các return về **exit node chung**, nếu không $E - N + 2$ cho kết quả sai (nhỏ hơn thực tế).

Buổi 4 sẽ đào sâu: dùng $V(G)$ chọn basis path có hệ thống, liên hệ với MC/DC và JaCoCo.

Tính $V(G)$ cho hai ví dụ

ageLabel()

- $N = 10, E = 12$ (có exit chung E)
- $V(G) = 12 - 10 + 2 = 4$
- Kiểm tra chéo: 3 node quyết định + 1 = **4** ✓
- Basis path = 4 đường P1–P4 (slide trước) $\Rightarrow$ tối thiểu **4 test case**

evaluate() – lỗi quyết định

- $N = 7, E = 8$
- $V(G) = 8 - 7 + 2 = 3$
- Kiểm tra chéo: 2 node quyết định + 1 = **3** ✓
- Basis path = 3 đường PAUSE / WARN / ALLOW $\Rightarrow$ tối thiểu **3 test case**

Câu hỏi

Nếu “mở” node 1 của evaluate() ra – tính cả 3 câu lệnh if gom tín hiệu S1, S2, S3 – thì $V(G)$ bằng bao nhiêu? (Slide sau trả lời.)

CFG đầy đủ của `evaluate()`: thêm 3 `if` gom tín hiệu

Toàn bộ method có **5 node quyết định**:

- S1: `if (req.onCall())`
- S2: `if (req.newPayee())`
- S3: `if (watchlistHit)` — thêm tín hiệu
- D1: `onCall && newPayee && hit`
- D2: `else if (watchlistHit)`

$$V(G) = 5 + 1 = 6$$

Mỗi `if` S1–S3 không có `else` $\Rightarrow$ nhánh F là cạnh “đi tắt” — muốn 100% BC phải có test với tín hiệu **tắt** lẫn **bật**.

Hệ quả cho thiết kế test

3 test {PAUSE, WARN, ALLOW} phủ 100% nhánh của **lỗi quyết định**, nhưng phủ hết nhánh S1, S2 hay không còn **tùy cách**

chọn input:

- Nếu WARN dùng `onCall=T`, `newPayee=F` và ALLOW dùng `onCall=T`, `newPayee=T`: nhánh F của S2 được phủ, nhưng nhánh F của S1 **chưa ai đi!**

Với cách chọn ở slide sau, {T1, T2, T4} đã đủ **100% BC**; T3 giúp bẻ phủ **tường minh, dễ truy vết**.

Bảng test case 100% BC cho evaluate() – toàn method

TC	onCall	newPayee	watchlist	Expected decision	signals
T1	T	T	T	PAUSE	cả 3 tín hiệu
T2	F	F	T	WARN	WATCHLIST_HIT
T3	T	T	F	ALLOW	ON_CALL, NEW_PAYEE
T4	F	F	F	ALLOW	rỗng

Kiểm tra độ phủ nhánh (5 quyết định $\times 2 = 10$ nhánh)

S1: T (T1,T3) / F (T2,T4) ✓ S2: T (T1,T3) / F (T2,T4) ✓ S3: T (T1,T2) / F (T3,T4) ✓
D1: T (T1) / F (T2–T4) ✓ D2: T (T2) / F (T3,T4) ✓ $\Rightarrow$ **100% BC với 4 test case.**

Đối chiếu với unit test THẬT trong repo

```
modules/interception/src/  
test/.../InterceptionServiceImplTest.java
```

5 test method — 4 test cho `evaluate()` khớp **đúng** bảng T1–T4 vừa thiết kế, cộng 1 test cho `checkAccount()`:

- 1 `evaluate_whenOnCallAndNewPayeeAndWatchlistHit_pauses...` — T1
- 2 `evaluate_whenWatchlistHitButNotOnCallNorNewPayee_warns` — T2
- 3 `evaluate_whenOnCallAndNewPayeeButNoWatchlistHit_...Allows` — T3
- 4 `evaluate_whenNoSignals_allowsAndPersistsEmptySignals` — T4
- 5 `checkAccount_delegatesToRepositoryScopedByTenant`

Nhận xét

Comment trong file test thật ghi rõ: *“Each branch of `evaluate(...)` is covered by a dedicated test method”* — dev đã thiết kế test theo đúng tư duy **độ phủ nhánh** ta vừa học. Mỗi test = một dòng của bảng test case.

Test thật T1 — nhánh PAUSE (rút gọn)

```
1 @Test
2 void evaluate_whenOnCallAndNewPayeeAndWatchlistHit_pauses() {
3     // Arrange: tại khoản dịch nam trong watchlist
4     when(watchRepository.findByTenantIdAndValue(
5         TENANT_ID, DEST_ACCOUNT))
6         .thenReturn(Optional.of(watchHit));
7     EvaluateRequest request = new EvaluateRequest(
8         userId, new BigDecimal("5000000"),
9         DEST_ACCOUNT, true, true); // onCall=T, newPayee=T
10
11     // Act
12     InterventionDecision d = service.evaluate(request);
13
14     // Assert: đi dùng đường PAUSE (node 3)
15     assertThat(d.decision()).isEqualTo("PAUSE");
16     assertThat(d.signals()).containsExactly(
17         "ON_CALL", "NEW_PAYEE", "WATCHLIST_HIT");
18 }
```

Test thật T4 — nhánh ALLOW, không tín hiệu (rút gọn)

```
1 @Test
2 void evaluate_whenNoSignals_allowsAndPersistsEmptySignals() {
3     // Arrange: không trung watchlist
4     when(watchRepository.findByTenantIdAndValue(
5         TENANT_ID, DEST_ACCOUNT))
6         .thenReturn(Optional.empty());
7     EvaluateRequest request = new EvaluateRequest(
8         userId, new BigDecimal("10"),
9         DEST_ACCOUNT, false, false); // mọi tín hiệu tắt
10
11     InterventionDecision d = service.evaluate(request);
12
13     assertThat(d.decision()).isEqualTo("ALLOW");
14     assertThat(d.signals()).isEmpty();
15     // Bước sự kiện được lưu để kiểm tra dữ liệu
16     verify(eventRepository).save(eventCaptor.capture());
17     assertThat(eventCaptor.getValue().getSignals()).isEmpty();
18 }
```

Test này phủ nhánh **F** của cả S1, S2, S3 — đúng vai trò của T4 trong bảng. Mock/captor sẽ học kỹ ở buổi 6.

Tự chạy bộ test thật

Lệnh chạy (không cần Docker)

```
cd digishield  
./gradlew test --tests "InterceptionServiceImplTest"
```

- Kỳ vọng: **5 test PASS** trong vài giây — unit test thuần Mockito, không cần database.
- Thử “phá”: sửa điều kiện trong `evaluate()` (ví dụ bỏ `&& watchlistHit`) rồi chạy lại — test nào FAIL? Nhánh nào đã bắt được lỗi?
- Buổi 4 sẽ dùng **JaCoCo** xác nhận bằng máy: `./gradlew testCodeCoverageReport`.

Vòng lặp trong luồng điều khiển — loop testing

Vấn đề

Cạnh quay lui của vòng lặp tạo **vô số** đường đi (0 lần, 1 lần, 2 lần, ... lặp). Không thể phủ hết mọi đường $\Rightarrow$ chọn các trường hợp **đại diện**.^[3]

Bộ test chuẩn cho vòng lặp đơn (số lần lặp tối đa n):

- 1 **0 lần** — nhảy qua vòng lặp
- 2 **1 lần** lặp
- 3 **2 lần** lặp
- 4 Số lần **điển hình** m ($2 < m < n$)
- 5 $n - 1$, n , và thử $n + 1$ lần (nếu ép được)

Lỗi vòng lặp hay bắt được

- Sai điều kiện dừng (off-by-one)
- Biến tích lũy khởi tạo sai
- Xử lý sai khi tập dữ liệu **rỗng**

Ví dụ: vòng lặp đếm hit trong StubAiClient.classify()

Code thật (rút gọn) — bộ phân loại phishing dự phòng khi không gọi Claude

```
1 public ClassificationView classify(String payload) {
2     String text = payload == null
3         ? "" : payload.toLowerCase(Locale.ROOT);
4     long threatHits = countHits(text, THREAT_SIGNALS);
5     if (threatHits > 0) {
6         double confidence = clamp(0.6 + 0.1 * threatHits);
7         return new ClassificationView("threat", confidence, ...);
8     }
9     // ... tương tự cho spam, cuối cùng: "clean", 0.92
10 }
11 // countHits tương đương vòng lặp đơn:
12 long hits = 0;
13 for (String s : signals) {           // lặp 10 lần cơ định
14     if (text.contains(s)) hits++;    // thay if bên trong
15 }
```

THREAT_SIGNALS = 10 từ khóa: "password", "urgent", "otp", "click here", ...

Áp dụng loop testing cho bộ đếm hit

TC	Payload	hits	Expected
L0	"hop qua bình thuong"	0	"clean", 0.92
L1	"nhap password ngay"	1	"threat", 0.7
L2	"password + otp"	2	"threat", 0.8
Lmax	chứa ≥ 5 từ khóa	≥ 5	"threat", 1.0 (clamp)

Điểm thú vị: biên của clamp

$0.6 + 0.1 \times 4 = 1.0$ — tại 4 hit confidence **chạm đúng biên** 1.0 (chưa cần chặn); từ **5 hit** trở lên giá trị vượt 1.0 và mới bị clamp cắt. Test Lmax (≥ 5 hit) vừa kiểm tra “nhiều lần lặp” vừa chạm nhánh `Math.min` bên trong `clamp()`.

Lưu ý: vòng `for` ở đây luôn lặp đủ 10 lần — các case L0/L1/L2/Lmax thay đổi **số lần nhánh `if` trong thân lặp kích hoạt** (số hit), tức một *biến thể* của loop testing, chứ không phải bộ 0/1/2/max **lần lặp** kinh điển ở slide 36. Vòng lặp có số lần lặp biến thiên đúng nghĩa: xem `benchmark()` ở Thực hành 2.

Test thật tương ứng: `StubAiClientTest (5 test)` trong `modules/ai`.

Kiểm thử luồng điều khiển KHÔNG phát hiện được gì?

Hạn chế

- **Code thiếu:** yêu cầu chưa được cài đặt thì không có node/nhánh nào để phủ! (VD: `evaluate()` chưa xét hạn mức `amount` — 100% BC vẫn không lộ)
- **Lỗi đặc tả:** code chạy đúng như dev viết, nhưng dev hiểu sai yêu cầu
- **Lỗi dữ liệu / biên:** 100% BC vẫn lọt lỗi $<$ vs $\leq$ tại biên
- Đường đi **bất khả thi** (infeasible path) làm mục tiêu 100% path bất khả đạt

Cách khắc phục

- Kết hợp **hộp đen** (EP, BVA, bảng quyết định — buổi 8) để soi từ phía yêu cầu
- Traceability: mỗi yêu cầu $\rightarrow$ ít nhất 1 test case
- Review chéo đặc tả và code

Ghi nhớ

Độ phủ đo “bộ test đã chạm bao nhiêu **code**”, không đo “phần mềm đúng bao nhiêu **yêu cầu**”.

Thực hành 1: StubAiClient.moderate() — code thật

Kiểm duyệt nội dung template phishing mô phỏng (modules/ai)

```
1 public ModerationView moderate(String content) {
2     String text = content == null
3         ? "" : content.toLowerCase(Locale.ROOT);
4     List<String> reasons = new ArrayList<>();
5     for (String word : UNSAFE_WORDS) { // 10 tu khoa
6         if (text.contains(word)) {
7             reasons.add("Noi dung chua tu khong an toan: "
8                 + word);
9         }
10    }
11    String verdict;
12    if (reasons.isEmpty()) {
13        verdict = "pass";
14    } else if (reasons.size() >= 2) {
15        verdict = "block";
16    } else {
17        verdict = "flag";
18    }
19    return new ModerationView(verdict, List.copyOf(reasons));
20 }
```

Thực hành 1 — yêu cầu (làm tại lớp, nhóm 2 người)

- 1 Vẽ CFG của `moderate()` (gộp vòng lặp `for` thành một cụm “đếm reasons” có node quyết định + cạnh quay lui).
- 2 Tính $V(G)$ bằng cả hai công thức — đối chiếu kết quả.
- 3 Thiết kế bộ test đạt **100% độ phủ nhánh**, điền bảng:

TC	content	reasons.size()	Expected
M1	“Hop thu thông báo lịch hop”	0	"pass"
M2	chứa đúng 1 từ khóa (VD "malware")	1	?
M3	chứa 2 từ khóa (VD "malware", "ddos")	2	?
M4	<code>content = null</code>	?	?

Câu hỏi thêm: M3 với `size()=2` là **on-point** của biên ≥ 2 — test off-point nào nên bổ sung? So kết quả với `StubAiClientTest` trong repo.

Thực hành 2:

AnalyticsServiceImpl.benchmark() — code thật

Điểm rủi ro trung bình theo phạm vi (modules/analytics)

```
1 public int benchmark(RiskScope scope) {  
2     UUID tenantId = TenantContext.requireUuid();  
3     List<RiskScore> scores = riskScoreRepository  
4         .findByTenantIdAndScope(tenantId, scope);  
5     if (scores.isEmpty()) {  
6         return 0;           // danh sách rỗng  
7     }  
8     long sum = scores.stream()  
9         .mapToInt(RiskScore::getValue).sum();  
10    return (int) (sum / scores.size());  
11 }
```

Yêu cầu:

- 1 Vẽ CFG, tính $V(G)$, liệt kê đường đi.
- 2 Thiết kế bộ test 100% BC — chú ý trường hợp **list rỗng** → **0** (nhánh chống chia cho 0) và áp dụng loop testing cho phần tính tổng: 0 / 1 / nhiều phần tử.
- 3 So sánh với AnalyticsServiceImplTest (9 test) trong repo.

Bài nộp — deadline: trước buổi 4

Bài cá nhân

- 1 Hoàn thiện 2 bài thực hành trên lớp (CFG + $V(G)$ + bảng test case cho `moderate()` và `benchmark()`).
- 2 Vẽ CFG **đầy đủ** của `evaluate()` (cả 5 node quyết định), tính $V(G)$, kiểm chứng bằng T1–T4 phủ 10/10 nhánh.

Bài nhóm — gắn với Bài tập lớn (30%)

Trong module DigiShield nhóm đã chọn ở buổi 2: chọn **2 method** có ít nhất 2 điểm quyết định mỗi method, rồi:

- Vẽ CFG (draw.io / tay, chụp ảnh), đánh số node
- Tính $V(G)$, liệt kê basis path
- Lập bảng test case đạt 100% độ phủ nhánh (input → expected)
- Đưa vào chương “Thiết kế test hộp trắng” của báo cáo BTL

Định dạng

Nộp `HoTen_MaSV_Buoi03.zip` (bài cá nhân) qua LMS; bài nhóm commit vào repo BTL của nhóm.

Câu hỏi ôn tập

- 1 100% branch coverage có suy ra 100% statement coverage không? Ngược lại? Cho ví dụ minh họa bằng `if` không có `else`.
- 2 Vẽ CFG cho method có 2 `if` nối tiếp (không lồng nhau, đều có `else`). Có bao nhiêu đường đi? $V(G)$ bằng bao nhiêu?
- 3 Vì sao khi method có nhiều `return`, ta phải thêm exit node chung trước khi áp dụng $V(G) = E - N + 2$?
- 4 Với `evaluate()`: một bộ test đạt 100% BC trên lỗi quyết định nhưng chưa đạt 100% BC trên toàn method — tình huống nào gây ra điều đó?
- 5 Bộ test tối thiểu cho một vòng lặp đơn gồm những trường hợp nào?
- 6 Kể 2 loại lỗi mà kiểm thử luồng điều khiển **không thể** phát hiện, và kỹ thuật nào bù đắp được.

Chuẩn bị cho buổi 4

Buổi 4 — Chương 3 (tiếp): hộp trắng nâng cao

- Độ phủ điều kiện (condition coverage) và **MC/DC** — mổ xẻ điều kiện phức hợp `onCall && newPayee && watchlistHit`
- Basis path testing có hệ thống với $V(G)$
- Kiểm thử luồng dữ liệu (def-use pairs)
- Đo độ phủ bằng máy: **JaCoCo** trên DigiShield

Việc cần làm trước

- Hoàn thành bài nộp buổi 3
- Clone và chạy được: `./gradlew test` (296 test PASS)
- Đọc trước Ammann & Offutt, chương 2 (Graph Coverage) — mục 2.1–2.2
- Suy nghĩ: điều kiện `A && B && C` cần bao nhiêu test để mỗi điều kiện con “tự chứng minh” ảnh hưởng của mình?

Tài liệu tham khảo

- [1] Slide bài giảng điện tử môn Kiểm thử Phần mềm.
- [2] Paul Ammann & Jeff Offutt. *Introduction to Software Testing*, Cambridge University Press, 2008. (Chương 2: Graph Coverage)
- [3] Glenford J. Myers. *The Art of Software Testing*, 2nd ed., Wiley, 2004.
- [4] ISTQB Foundation Level Syllabus, version 4.0, 2023. <https://istqb.org/>
- [5] Thomas J. McCabe. “A Complexity Measure.” *IEEE Transactions on Software Engineering*, SE-2(4), 1976.

Tóm tắt buổi 3

Lý thuyết:

- Hộp trắng: thiết kế test từ **cấu trúc code**
- CFG: node, cạnh, node quyết định, exit chung
- Độ phủ: $\text{Statement} \subset \text{Branch} \subset \text{Path}$
- $V(G) = E - N + 2 = \text{số quyết định} + 1 = \text{số test tối thiểu}$
- Loop testing: 0 / 1 / 2 / điển hình / max
- Hạn chế: không thấy code thiếu, lỗi đặc tả

Trên DigiShield:

- `ageLabel()`: CFG 10 node, $V(G) = 4$, 4 test
- `evaluate()`: lỗi $V(G) = 3$, toàn method $V(G) = 6$, 4 test phủ 10/10 nhánh
- Đối chiếu 5 test thật của `InterceptionServiceImplTest`
- Loop testing trên bộ đếm hit của `StubAiClient`

Buổi tới

Condition coverage, MC/DC, luồng dữ liệu, đo độ phủ bằng JaCoCo.